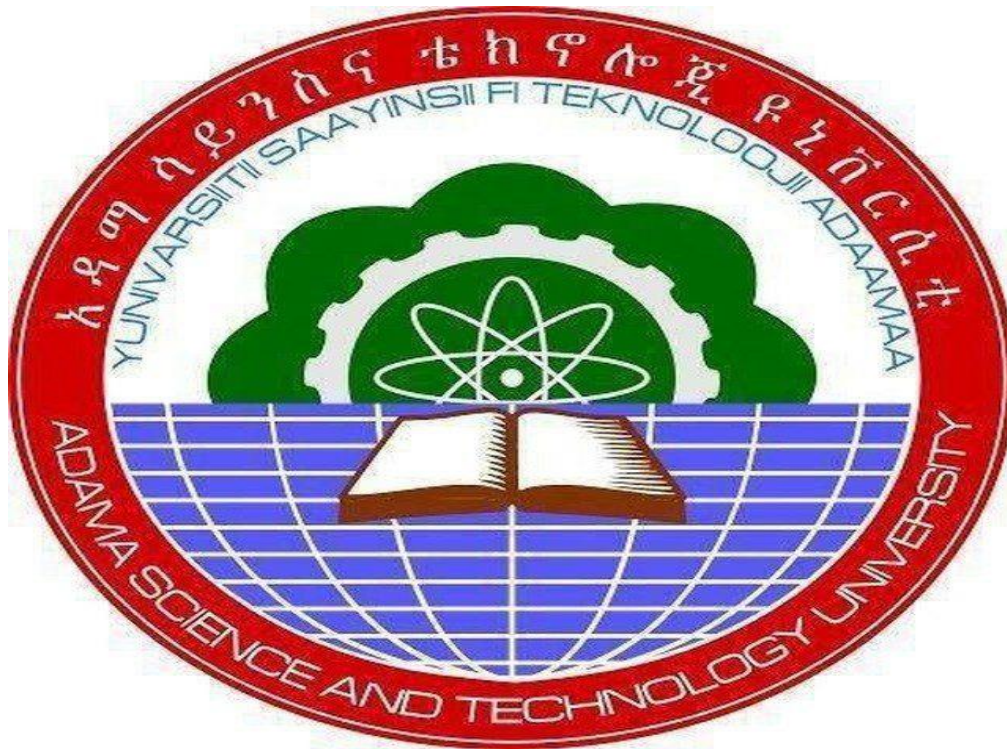


Adama Science and Technology University



School of Electrical Engineering and Computing (SOEEC/CSE)

Department:- Computer Science and Engineering

Course Name : Programming Languages

Course code. :- CSEg 4306

GROUP MEMBERS r.	IDNO.	SECTION
1. Segni Nadew	UGR/312051/ 15.....	1
2. Deressa Fufa.....	UGR/30410/15	1

Submitted to Ms Hiwot Habtamu

Question 1: Parameter Passing Modes in Subroutines

Parameter passing defines how data flows between a caller and a called subroutine. The three classical modes are **call-by-value**, **call-by-reference**, and **call-by-value/result**. Each differs in how formal parameters relate to actual parameters.

1.1 Call-by-Value

Mechanism: When a subroutine is called, the value of the actual parameter is *copied* into the formal parameter. The formal parameter is an independent local variable inside the subroutine. Any modification to the formal parameter has **no effect** on the actual parameter in the caller's scope.

Interaction: The actual parameter is read once at call time. Subsequent writes to the formal parameter stay local.

Code Example (C):

```
#include <stdio.h>

void increment(int y) {    //y is a copy of the actual argument
    y = y + 1;
    printf("Inside: y = %d\n", y);    // prints 6
}

int main() {
    int x = 5;
    increment(x);
    printf("After call: x = %d\n", x);    // prints 5-unchanged
    return 0;
}
```

Advantage: Safety—the caller's data cannot be accidentally modified by the subroutine, which makes code easier to reason about.

Disadvantage: Inefficiency for large data structures, because the entire object must be copied even when the subroutine only needs to read it.

1.2 Call-by-Reference

Mechanism: Instead of copying the value, the subroutine receives a reference (alias or pointer) to the actual parameter's storage location. Any assignment to the formal parameter directly modifies the original variable in the caller.

Interaction: Formal and actual parameters share the same memory cell throughout the subroutine's execution.

Code Example (C++):

```
#include <iostream>
using namespace std;

void increment(int& y) {    //y is an alias for the actual argument
    y = y + 1;
    cout << "Inside: y = " << y << endl;    // prints 6
}
```

```

}

int main() {
    int x = 5;
    increment(x);
    cout << "After call: x = " << x << endl; //prints 6-changed
    return 0;
}

```

Advantage: Efficient for large structures (no copy), and enables subroutines to return multiple values through their parameters.

Disadvantage: Aliasing hazards – unintended side effects can make programs harder to understand and debug; callers must be careful that their variables will be mutated.

1.3 Call-by-Value/Result (Copy-In/Copy-Out)

Mechanism: A two-phase strategy. On entry (*copy-in*) the actual parameter's value is copied into the formal parameter, just like call-by-value. On return (*copy-out*) the final value of the formal parameter is written back into the actual parameter. Intermediate changes therefore do NOT appear in the caller until the subroutine returns.

Interaction: Independent during execution; synchronised at return.

Code Example (Ada-style pseudo-C):

```

/*Ada'inout'parameter-semanticsillustratedinC*/ #include
<stdio.h>

void swap_demo(int* y_in, int* y_out, int init_val) {
    /*copy-inphase:y=init_val*/ int y =
    init_val;
    y = y + 10;          /* operations on local copy */
    *y_out = y;          /* copy-out phase on return */
}

int main() {
    int x = 5;
    int out_val;
    swap_demo(&x, &out_val, x); /*passvalueofx,writebacktoout_val*/ x =
    out_val;                /* simulates copy-out into x */
    printf("After call: x = %d\n", x); /* prints 15 */
    return 0;
}

```

Advantage: Avoids aliasing problems during execution, making subroutine bodies easier to analyse formally.

Disadvantage: Requires two memory accesses (copy-in and copy-out), and results can differ from call-by-reference when aliasing exists (e.g., passing the same variable twice).

1.4 Analysis of the Given Code Snippet

```

var x: integer;
procedure foo(y: integer);
begin
    y := 3;
    print(x);
end;

```

```

end;
begin
  x := 2;
  foo(x);
  print(x);
end.

```

We trace through each passing mode:

Mode	y := 3 changes...	print(x) inside foo	print(x) after foo
Call-by-Value	Only local copy. x remains 2.	Prints 2 (x unaffected)	Prints 2 (x still 2)
Call-by-Reference	x directly (y is alias for x). x becomes 3.	Prints 3 (x already changed)	Prints 3 (x = 3)
Call-by-Value/Result	Only local copy during execution. Copy-out writes 3 to x on return.	Prints 2 (copy-out not done yet)	Prints 3 (copy-out applied on return)

Summary: The key distinction is visibility of mutations. Call-by-value isolates the callee completely; call-by-reference makes every change immediately visible; call-by-value/result defers visibility until return, producing results that can differ from call-by-reference when aliasing is present.

Question 2: Calling Sequence in RISC Architecture

A **calling sequence** is the protocol through which control and data are transferred between a caller and a callee. In RISC (Reduced Instruction Set Computing) architectures such as MIPS, ARM, or RISC-V, the convention is split into clearly defined **caller responsibilities** and **callee responsibilities**.

2.1 Caller Responsibilities (Before the Call)

- **Save caller-saved registers:** Any register whose value the caller needs after the call and that is not callee-saved must be saved by the caller (typically to the stack or other caller-saved slots).
- **Evaluate and pass arguments:** Arguments are placed in the designated argument registers (e.g., a0–a7 in RISC-V, x0–x7 in ARM64, \$a0–\$a3 in MIPS). Arguments beyond the register count are pushed onto the stack in right-to-left order.
- **Save the return address:** The call instruction (JAL in RISC-V, BL in ARM, JAL in MIPS) stores the return address in the link register (ra / lr / \$ra).
- **Jump to callee:** Control transfers to the first instruction of the called subroutine.

2.2 Callee Responsibilities (On Entry – Prologue)

- **Save the frame pointer (FP/s0):** The current frame pointer is pushed onto the stack so that the caller's frame can be restored.
- **Establish a new frame pointer:** The frame pointer is set to the current stack pointer, marking the base of the new activation record.
- **Allocate stack frame:** The stack pointer (SP) is decremented by the size of the local data area (local variables, temporaries, spilled registers).
- **Save callee-saved registers:** Any callee-saved registers (e.g., s0–s11 in RISC-V) that the callee will use must be preserved at this point.

2.3 Callee Responsibilities (On Return – Epilogue)

- **Place return value(s):** Result(s) are placed in the return-value registers (e.g., a0–a1 in RISC-V, x0–x1 in ARM64).
- **Restore callee-saved registers:** All saved callee-saved registers are reloaded from the stack.
- **Deallocate frame:** The stack pointer is restored to its original value (often by copying the frame pointer back to SP).
- **Restore frame pointer:** The caller's frame pointer is reloaded.
- **Return to caller:** A RET instruction (or JR \$ra in MIPS) transfers control back using the saved return address in the link register.

2.4 Illustrative RISC-V Assembly Sketch

```
# Caller side (calling add_two(a, b))
addi a0, x0, 3      # first argument: 3
addi a1, x0, 4      # second argument: 4
```

```

jal    ra, add_two    # call; ra ←return address

#add_twoprologue(callee)
add_two:
    addi    sp, sp, -8    #allocate 8 bytes on stack
    sw      ra, 4(sp)    # save return address
    sw      s0, 0(sp)    # save callee-saved s0
    addi    s0, sp, 8    #establish frame pointer

    add     a0, a0, a1    # compute sum →return in a0

    # add_two epilogue
    lw      s0, 0(sp)    # restore s0
    lw      ra, 4(sp)    # restore return address
    addi    sp, sp, 8    # deallocate frame
    ret     # return to caller (pc ←ra)

```

The RISC philosophy keeps instructions uniform and simple; the calling convention compensates by relying on software discipline (register-save protocols) rather than complex hardware mechanisms. This exposes more work to the compiler but allows greater optimisation freedom.

Question 3: Exception Handling in Subroutines

3.1 What Is an Exception?

An **exception** is an abnormal or unexpected event that disrupts the normal sequential flow of a program during execution. It signals that a subroutine (or the system) has encountered a condition it cannot handle in its normal logic. Exception handling provides a structured, language-supported mechanism for detecting such conditions, transferring control to a designated handler, and optionally resuming or terminating the program cleanly.

Two examples of exceptions:

1. **Arithmetic overflow/division by zero:** Occurs when an integer arithmetic operation produces a result too large for its type, or when a division by zero is attempted. This is typically raised by the hardware or runtime system.
2. **User-defined condition (custom exception):** A programmer may define and throw an exception class to represent domain-specific errors, such as *InsufficientFundsException* in a banking application, when a withdrawal exceeds the available balance.

3.2 Lexical Binding of Exception Handlers

In C++ (and Ada), exception handlers are **lexically bound** to a block of code via the **try-catch** construct. This means the scope of the handler is determined by its textual position in the source code, not by runtime call chains. When an exception is thrown inside a try block, the runtime searches for a matching catch clause in the immediately enclosing try block first, then walks up the call stack (unwinding) until a handler is found or the program terminates.

```
#include <iostream>
#include <stdexcept>
using namespace std;

double safe_divide(int numerator, int denominator) { if
    (denominator == 0)
        throw runtime_error("Division by zero detected");
    return static_cast<double>(numerator) / denominator;
}

int main() { try
{
    // guarded block
    cout << safe_divide(10, 2) << endl;    //OK: prints 5
    cout << safe_divide(7, 0) << endl;    //throws runtime_error
    cout << "This line is never reached" << endl;
    // end of guarded block
}
catch (const runtime_error& e) {           //handler lexically bound here
    cerr << "Caught exception: " << e.what() << endl;
    // Handler can: recover, clean up, or terminate
}
catch (...) {                             //catch-all handler
    cerr << "Unknown exception occurred" << endl;
}
return 0;
```

```
}
```

3.3 Three Operations an Exception Handler May Perform

- 1. Recover:** The handler corrects the erroneous condition and allows execution to continue (either by resuming after the try block or by retrying the operation). Example: replace a bad input with a default value and proceed.
- 2. Cleanup (partial recovery):** The handler cannot fully recover but releases acquired resources (close files, free memory, release locks) before re-throwing the exception or throwing a different one to higher-level handlers.
- 3. Terminate with an error message:** The handler determines that continuation is impossible, logs or prints a diagnostic message, and terminates the program (or the current thread) gracefully, avoiding undefined behaviour or data corruption.

3.4 Group Code Example – Arithmetic Overflow / Division Handling

```
#include <iostream>
#include <stdexcept>
#include <limits>
using namespace std;

// Custom exception class for division errors class
DivisionError : public runtime_error { public:
    explicit DivisionError(const string& msg) : runtime_error(msg) {}
};

// Custom exception for overflow
class OverflowError : public runtime_error { public:
    explicit OverflowError(const string& msg) : runtime_error(msg) {}
};

// Safe division that checks for divide-by-zero and overflow
double safe_divide(long long numerator, long long denominator) {
    if (denominator == 0)
        throw DivisionError("Cannot divide by zero.");

    // Check potential integer overflow before the division
    if (numerator == numeric_limits<long long>::min() && denominator == -1)
        throw OverflowError("Integer overflow: result exceeds long long range.");

    return static_cast<double>(numerator) / denominator;
}

int main()
{
    // Test 1: normal division
    try {
        cout << "10 / 2 = " << safe_divide(10, 2) << endl;
    }
    catch (const DivisionError& e) { cerr << "Division error: " << e.what() << endl; }
    catch (const OverflowError& e) { cerr << "Overflow error: " << e.what() << endl; }

    // Test 2: divide by zero → recover with default value
    try {
```

```

        cout << "7 / 0 = " << safe_divide(7, 0) << endl;
    }
    catch (const DivisionError& e) {
        cerr << "[Caught] " << e.what() << endl;
        //RECOVER:substituteasafedefault double
        default_val = 0.0;
        cout << "Recovering: using default value " << default_val << endl;
    }

    // Test 3: overflow →terminate gracefully
    try {
        long long min_ll = numeric_limits<long long>::min();
        cout << "MIN / -1 = " << safe_divide(min_ll, -1) << endl;
    }
    catch (const OverflowError& e) {
        cerr << "[Caught] " << e.what() << endl;
        // TERMINATE GRACEFULLY: log and exit
        cerr<<"Criticalarithmeticerror.Programcannotcontinue."<<endl; return
        1;
    }

    return 0;
}

```

Discussion: In Test 2 the handler recovers by substituting a default result and continuing execution. In Test 3 the handler chooses graceful termination: it logs a descriptive message and exits with a non-zero status code, allowing the operating system or a parent process to detect the failure. A clean-up approach would additionally flush open file streams or released database connections before exiting.

Question 4: Encapsulation and Inheritance in OOP

4.1 Encapsulation

Encapsulation is the principle of bundling data (attributes) and the operations that act on that data (methods) into a single unit (a class), while restricting direct external access to the internal state. It separates the *interface* (what a class does) from the *implementation* (how it does it), allowing the implementation to change without affecting client code.

C++ Visibility Modifiers:

Modifier	Accessible by	Typical Use
public	Any code that can see the class	Interface – methods and constants the client needs
protected	Class itself and all derived classes	Implementation details that subclasses may need to override or see
private	Only the class itself (not even derived classes)	Private internal state and helper methods (default in C++ class)

```
class BankAccount {
private:
    double balance;           // hidden internal state
    void audit_log(double amount); // private helper

protected:
    string owner_name;        // accessible by derived classes (e.g. SavingsAccount)

public:
    BankAccount(const string& name, double initial)
        : owner_name(name), balance(initial) {}

    void deposit(double amount) { // public interface
        if (amount > 0) balance += amount;
    }
    bool withdraw(double amount) {
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }
    double get_balance() const { return balance; } // read-only accessor
};
```

How encapsulation reduces conceptual load: A user of `BankAccount` does not need to know how the balance is stored or validated. They interact only with `deposit`, `withdraw`, and `get_balance`. This shrinks the mental model required to use the class and prevents accidental corruption of internal state.

4.2 Inheritance

Inheritance allows a *derived class* (subclass) to acquire the attributes and methods of a *base class* (superclass), enabling code reuse and the modelling of 'is-a' relationships. In C++, a class inherits using the syntax `class Derived : [access-specifier] Base`.

Inheritance Access Specifiers in C++:

Specifier	public members of Base become...	protected members of Base become...
public	public in Derived	protected in Derived
Specifier	public members of Base become...	protected members of Base become...

protected	protected in Derived	protected in Derived
private	private in Derived	private in Derived

```
#include <iostream>
#include <cmath>
using namespace std;

//Baseclass
class Shape{
protected:
    string color;
public:
    Shape(const string& c) : color(c) {}
    virtual double area() const = 0;    //purevirtual
    virtual void describe() const {
        cout << "Shape of color: " << color << endl;
    }
};

//Derived class-public inheritance(Circle IS-A Shape) class
Circle : public Shape {
private:
    double radius;
public:
    Circle(const string& c, double r) : Shape(c), radius(r) {}

    // Override base method
    double area() const override {
        return M_PI*radius*radius;
    }

    void describe() const override {
        cout << "Circle [color=" << color
            << ", radius=" << radius
            << ", area=" << area() << "]" << endl;
    }
};

// Another derived class
class Rectangle:public Shape{ private:
    double width,height;
public:
    Rectangle(const string& c, double w, double h)
        : Shape(c), width(w), height(h) {}

    double area()const override{return width*height;} void

    describe() const override {
        cout << "Rectangle [color=" << color
            << ", " << width << "x" << height
            << ", area=" << area() << "]" << endl;
    }
};

int main() {
```

```
Circle    c("red",  5.0);
Rectangle r("blue", 4.0, 6.0);

c.describe();    //Circle[color=red,radius=5,area=78.54]
r.describe();    // Rectangle [color=blue, 4x6, area=24]
return 0;
}
```

How inheritance promotes code reusability: Both Circle and Rectangle inherit the color attribute and the describe interface from Shape. Common behaviour is written once in the base class; derived classes provide only what is specific to them. Adding a new shape (e.g., Triangle) requires writing only its unique logic, not repeating the common infrastructure.

Question 5: Dynamic Method Binding and Virtual Functions

5.1 Dynamic Method Binding and Subtype Polymorphism

Dynamic method binding (also called *late binding* or *runtime dispatch*) is the mechanism by which the correct method implementation is selected **at runtime**, based on the actual type of the object, rather than at compile time based on the declared (static) type of the reference or pointer.

This enables **subtype polymorphism**: a variable declared as a pointer or reference to a base class can transparently hold objects of any derived class, and method calls on that variable invoke the most-derived (overriding) implementation. This is a cornerstone of the Open/Closed Principle – new behaviour is added by creating new subclasses rather than modifying existing code.

5.2 C++ Virtual Functions and the vtable

C++ implements dynamic binding through a **virtual table (vtable)** – a per-class table of function pointers, one for each virtual method. Every object of a class with at least one virtual method contains a hidden pointer (vptr) to its class's vtable. When a virtual method is called through a pointer/reference, the runtime dereferences the vptr, looks up the correct function pointer in the vtable, and calls it.

vtable Diagram – Person and Student:

Person vtable		Student vtable
[0] →Person::print_mailing_label()	→overridden→	[0] →Student::print_mailing_label()
[1] →Person::~Person() (destructor)		[1] →Student::~Student() (destructor)

When a *Person** actually points to a *Student* object, its *vptr* points to the **Student vtable**. Calling *ptr->print_mailing_label()* therefore dispatches to *Student::print_mailing_label()*, not the base class version.

5.3 Code Example – Polymorphic Behaviour

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

//Base class with virtual method
class Person {
protected:
    string name;
    string address;
public:
    Person(const string& n, const string& a) : name(n), address(a) {}

    //Virtual method - will be dispatched dynamically
    virtual void print_mailing_label() const {
        cout<<"===MailingLabel==="<<endl; cout
            << name << endl;
    }
};
```

```

        cout << address << endl;
    }

    virtual ~Person() = default;    // virtual destructor is essential
};

//Derivedclass-overridesprint_mailing_label class
Student : public Person {
private:
    stringstudent_id;
    string university;
public:
    Student(const string& n, const string& a,
            conststring&id,conststring&uni)
        : Person(n, a), student_id(id), university(uni) {}

    //Override:addsstudent-specificinformation void
    print_mailing_label() const override {
        cout<<"===StudentMailingLabel==="<<endl; cout <<
            name                << endl;
        cout << university      <<"(ID:"<<student_id<<)"<<endl; cout <<
            address              << endl;
    }
};

// Another derived class
classProfessor:publicPerson{ private:
    stringdepartment;
public:
    Professor(const string& n, const string& a, const string& dept)
        : Person(n, a), department(dept) {}

    void print_mailing_label() const override {
        cout << "=== Professor Mailing Label ===" << endl;
        cout<<"Prof."<<name<<","<<department<<endl; cout <<
            address              << endl;
    }
};

int main() {
    //Polymorphiccontainer-allaccessedviaPerson*
    vector<Person*> people = {
        new Person    ("Alice Green",    "12OakStreet,Adama"),
        new Student   ("Bob Tadesse",    "45 Campus Rd, Adama",
            "CSE/0023/15",    "AdamaScienceandTechnologyUniversity"), new
        Professor("Dr. Lemma S.",    "ASTU Main Campus, Adama",
            "Computer Science and Engineering"),
    };

    //Dynamicdispatch:thecorrectoverrideiscalledforeachobject for
    (const Person* p : people) {
        p->print_mailing_label();    //resolvedatruntimeviaavtable cout
        << endl;
    }

    // Clean up

```

```
        for(Person*p:people)deletep; return  
        0;  
    }
```

Expected Output:

```
===MailingLabel=== Alice  
Green  
12 Oak Street, Adama
```

```
===StudentMailingLabel=== Bob  
Tadesse  
Adama Science and Technology University (ID: CSE/0023/15)  
45 Campus Rd, Adama
```

```
=== Professor Mailing Label ===  
Prof.Dr.LemmaS.,ComputerScienceandEngineering ASTU  
Main Campus, Adama
```

Analysis: Even though each element is accessed through a `Person*`, the runtime correctly dispatches `print_mailing_label()` to the actual object type—`Person`, `Student`, or `Professor`. This is dynamic binding in action. Without `virtual`, all three calls would invoke `Person::print_mailing_label` (static binding), demonstrating why `virtual` is essential for polymorphism in C++.